

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Домашнее задание №5

Межсервисное взаимодействие через RabbitMQ

по дисциплине «Программирование серверной логики»

Вариант: Сайт для поиска работы

Студент: Гельм Даниил

Группа: К3340 / БР1.1

Санкт-Петербург
2026

Содержание

1. Цель работы
2. Исходная архитектура
3. Настройка RabbitMQ
4. Реализация обмена сообщениями
5. Формат событий
6. Запуск и проверка
7. Выводы

1. Цель работы

Цель — подключить RabbitMQ к микросервисной системе Job Search Platform (JP2) и реализовать асинхронное межсервисное взаимодействие через очередь сообщений.

Синхронные HTTP-вызовы между сервисами сохранены для критических операций; RabbitMQ используется для событий, не блокирующих основной HTTP-ответ клиенту.

2. Исходная архитектура

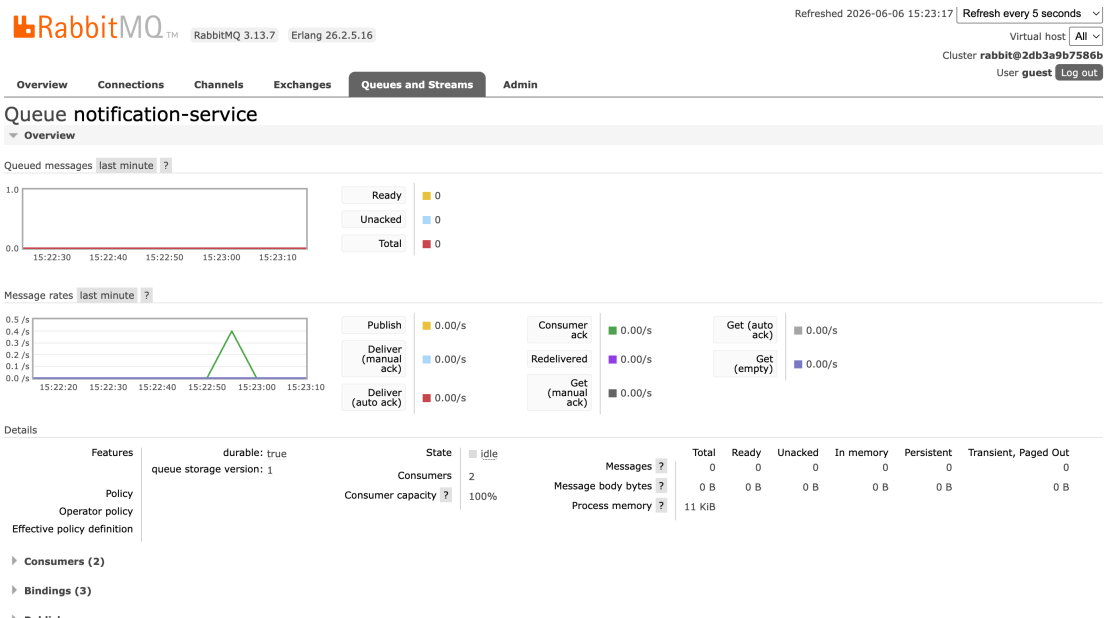
База — labs/lab2: API Gateway и 7 микросервисов. Application Service обрабатывает создание отклика и смену статуса. После ДЗ5 добавлен Notification Service — подписчик на события RabbitMQ.

3. Настройка RabbitMQ

RabbitMQ добавлен в docker-compose.yml проекта lab2:

```
rabbitmq:
  image: rabbitmq:3-management-alpine
  ports:
    - "5672:5672"
    - "15672:15672"
```

Параметр	Значение
AMQP	localhost:5672
Management UI	http://localhost:15672 (guest/guest)
Exchange	jobsearch.events (topic)
Очередь	notification-service



4. Реализация обмена сообщениями

Publisher: Application Service. После успешной записи в БД публикует событие в exchange `jobsearch.events` с routing key `application.created` или `application.status_changed`.

Consumer: Notification Service (порт 3008). Подписан на обе routing key, логирует события и хранит последние в памяти. Эндпоинт `GET /notifications/recent` — для проверки полученных событий.

Компонент	Файл	Роль
Publisher	labs/lab2/shared/rabbitmq.ts	Подключение и publish
Producer	services/application/src/index.ts	Отправка событий
Consumer	services/notification/src/index.ts	Приём и обработка

```
[nt] [notification] слушаю очередь notification-service
[nt] Notification Service http://localhost:3008
[app] Application Service http://localhost:3006
[fav] Favorites Service http://localhost:3007
[gw] API Gateway http://localhost:3000
[co] Company Service http://localhost:3002
[res] Resume Service http://localhost:3004
[auth] Auth Service http://localhost:3001
[vac] Vacancy Service http://localhost:3005
[sk] Skills Service http://localhost:3003
[nt] [notification] статус отклика #6: sent -> viewed
```

5. Формат событий

application.created:

```
{
  "type": "application.created",
  "applicationId": 1,
  "vacancyId": 5,
  "resumeId": 12,
  "applicantUserId": 7,
  "createdAt": "2026-06-06T12:00:00.000Z"
}
```

application.status_changed:

```
{
  "type": "application.status_changed",
  "applicationId": 1,
  "vacancyId": 5,
  "resumeId": 12,
  "oldStatus": "sent",
  "newStatus": "viewed",
  "changedAt": "2026-06-06T12:05:00.000Z"
}
```

6. Запуск и проверка

```
cd labs/lab2
npm run docker:up
npm run dev
# Postman-сценарий ДЗ3 через localhost:3000
```

```
curl http://localhost:3008/notifications/recent
```

The screenshot shows a web application interface for testing a job search scenario. The main panel displays the 'Run results' for 'DZ3 — сценарий (поиск работы)' which was executed today at 03:15:30 PM. The results table shows a successful run with 31 passed tests, 0 failures, and 0 errors. The tests include various API endpoints like 'Отклик ZU1', 'Статус sent', '13 Add favorite', '14 List favorites', '15 My applications', and '16 Employer update application status'. Each test result is marked as 'PASS' and includes details like response time and size.

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	DZ3 Local (Job Search)	1	2s 256ms	31	0	-

Summary: All 31 tests Passed. 31 Passed, 0 Failed, 0 Skipped, 0 Errors.

Test Results:

- POST Отклик ZU1: PASS
- POST Статус sent: PASS
- POST Комплексный сценарий > 13 Add favorite: PASS (201 ms, 37 B)
- GET Комплексный сценарий > 14 List favorites: PASS (200 ms, 13 ms, 561 B)
- GET Комплексный сценарий > 15 My applications: PASS (200 ms, 13 ms, 421 B)
- PATCH Комплексный сценарий > 16 Employer update application status: PASS (200 ms, 11 ms, 421 B)

The screenshot shows a terminal window with the command `curl http://localhost:3008/notifications/recent` and its output. The output is a JSON array of notification objects, including application status changes and new applications.

```
daniilgelm@m1 ~ % curl http://localhost:3008/notifications/recent
[{"type":"application.status_changed","applicationId":5,"vacancyId":5,"resumeId":5,"oldStatus":"sent","newStatus":"viewed","changedAt":"2026-06-06T12:15:32.963Z"}, {"type":"application.created","applicationId":5,"vacancyId":5,"resumeId":5,"applicantUserId":11,"createdAt":"2026-06-06T09:15:32.684Z"}, {"type":"application.status_changed","applicationId":4,"vacancyId":4,"resumeId":4,"oldStatus":"sent","newStatus":"viewed","changedAt":"2026-06-06T12:14:13.092Z"}, {"type":"application.created","applicationId":4,"vacancyId":4,"resumeId":4,"applicantUserId":9,"createdAt":"2026-06-06T09:14:12.918Z"}]
```

7. Выводы

RabbitMQ подключён к микросервисной системе. Application Service публикует события при создании отклика и смене статуса. Notification Service асинхронно получает и обрабатывает сообщения. HTTP API для клиента не изменился.